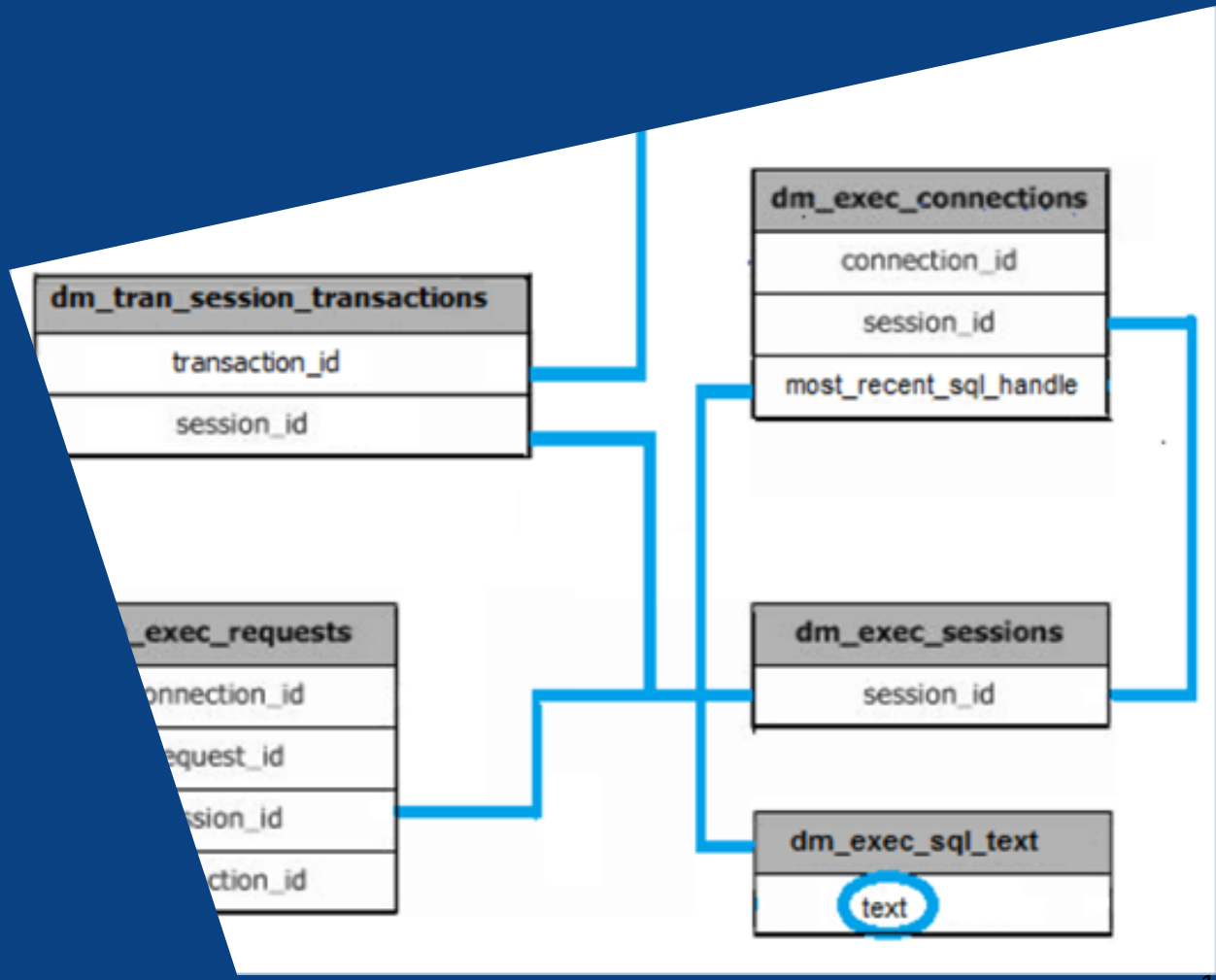
THE UNIVERSITY OF MELBOURNE

Transactions and Concurrency

Database Systems & Information Modelling
INFO90002



Week 7 – Database transactions

Presented by Greg Wadley
Content by: Dr Tanya Linden, Dr Renata Borovica-Gajic, David Eccles, Dr Simon D'Alfonso, Dr Greg Wadley

This Lecture Objectives

Why we need user-defined transactions

Properties of transactions

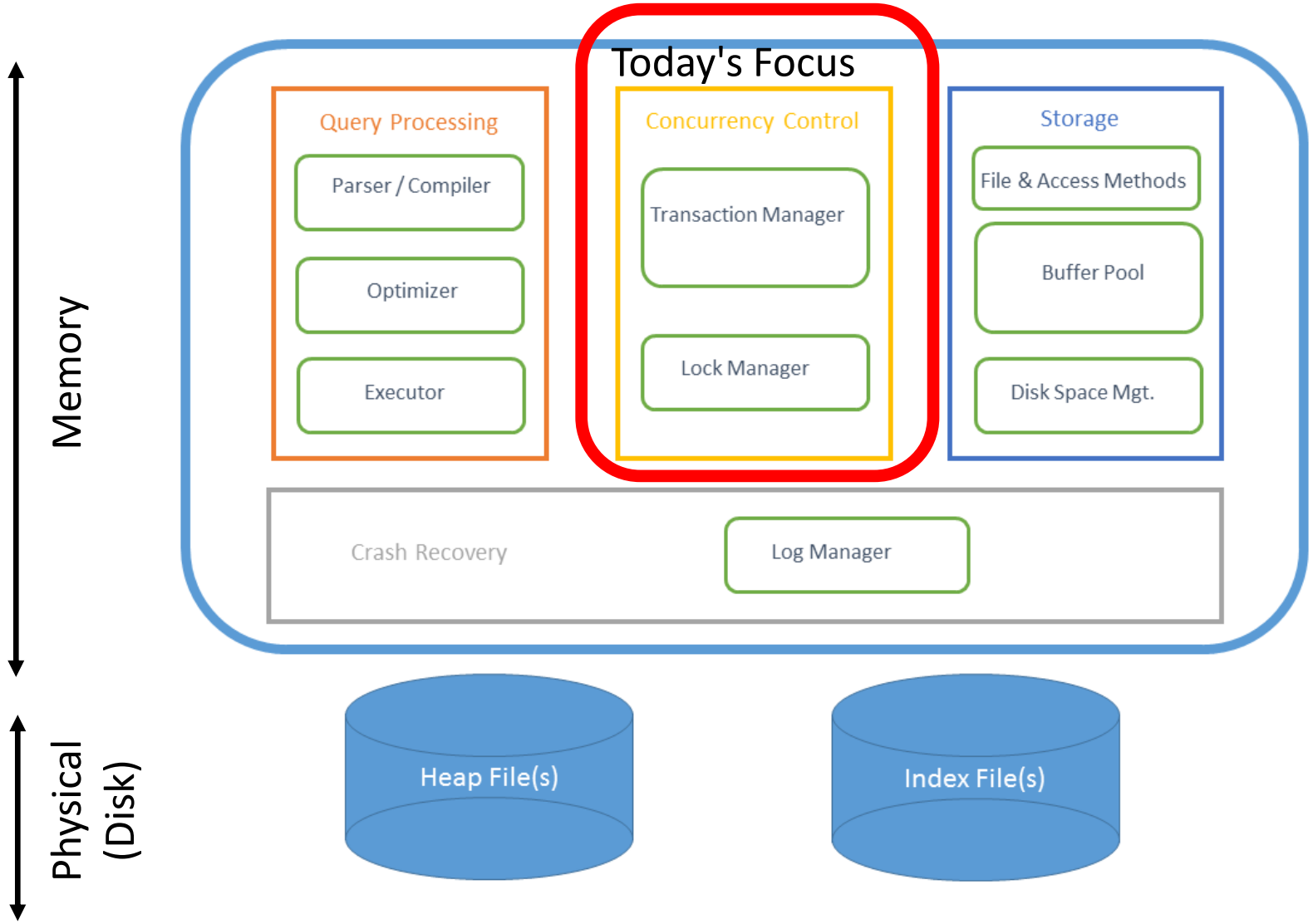
How to use transactions

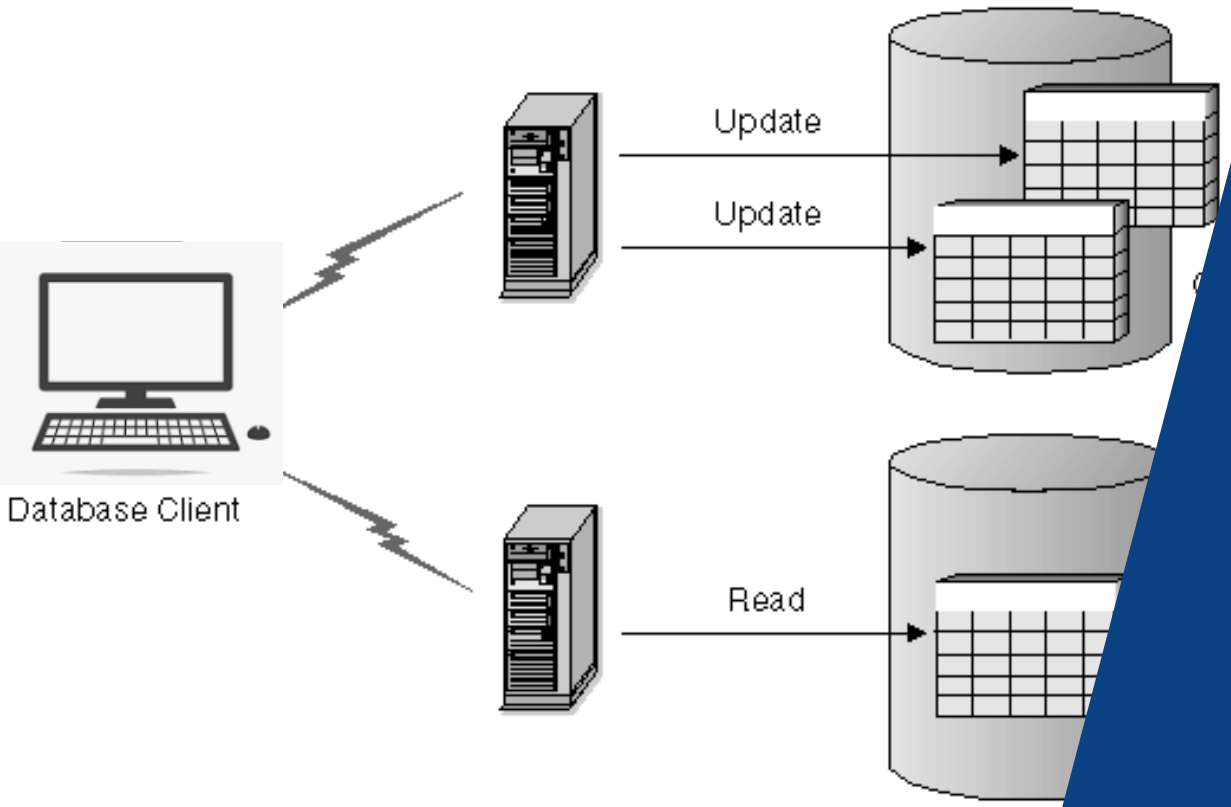
Concurrent access to data

Locking and deadlocking

Transaction's role in database recovery

Components of a DBMS





Transactions

Why we need database transactions

Business Transactions

Example “business transactions”:

- Placing order: Insert one row in *Order* table, then several in *OrderItem* table
- Funds transfer: Check amount < balance. If so, subtract amount from one row in BankAccount table, then add amount to another row. Record details in Transactions table
- Sending monthly statements: For all rows in Customer table, generate and send out monthly statements

Each requires several distinct database operations …

- Example: move money from savings account to credit card account
  - Accept inputs from user (via ATM, internet banking or mobile app)
  - Select balance from savings account
  - Is there enough money to withdraw? If so:
    - Update savings account balance = balance – withdrawal
    - Update credit card balance = balance + withdrawal
  - If no errors encountered, end successfully

What is a (database) Transaction?

A logical unit of work that must either be entirely completed or aborted (indivisible, atomic)

DML statements are already atomic *

RDBMS also allows for *user-defined* transactions

These are a sequence of DML statements, such as

- a series of UPDATE statements to change values
- a series of INSERT statements to add rows to tables
- DELETE statements to remove rows

Transactions will be treated as atomic

A successful transaction changes the database from one consistent state to another

- All data integrity constraints are satisfied

* In most popular relational DBMS

Why do we need Transactions?

Transactions solve TWO problems:

1. users need the ability to define a *unit of work*

2. *concurrent access* to data by >1 user or program

A transaction should not include tasks that are not relevant to each other.

For example, updating *quantity in stock* and updating *prices* for products to go on sale would not be in the same transaction.

Problem 1: Unit of work

- Single DML or DDL command (implicit transaction)
  - Example: update 700 records, but database crashes after 200 records processed
  - Restart server: you will find no changes to any records
  - Changes are “all or none”
- Multiple statements (user-defined transaction)
  - START TRANSACTION; (or, ‘BEGIN’)
    - SQL statement;
    - SQL statement;
    - SQL statement;
    - …
  - COMMIT; (commits the whole transaction, i.e. saves changes permanently)
    - Or ROLLBACK (to undo everything)

SQL keywords: **BEGIN; START TRANSACTION; COMMIT, ROLLBACK**

Business transactions as units of work

Each transaction consists of several SQL statements, embedded within a larger application program

Transaction needs to be an indivisible unit of work

- "Indivisible" means that either the whole job gets done, or none gets done:
  - if an error occurs, we don't leave the database with the job half done, in an inconsistent state

In the case of an error:

- Any SQL statements already completed must be reversed
- Show an error message to the user
- When ready, the user can try the transaction again
- This is briefly annoying – but inconsistent data is disastrous

Demo: Transaction as unit of work

```
mysql> SELECT * FROM account;
+----+---------+
| id | balance |
+----+---------+
|  1 |     100 |
|  2 |     200 |
+----+---------+
2 rows in set (0.00 sec)

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.01 sec)

mysql> UPDATE account SET balance=999;
Query OK, 2 rows affected (0.01 sec)
Rows matched: 2  Changed: 2  Warnings: 0

mysql> SELECT * FROM account;
+----+---------+
| id | balance |
+----+---------+
|  1 |     999 |
|  2 |     999 |
+----+---------+
2 rows in set (0.00 sec)

mysql> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT * FROM account;
+----+---------+
| id | balance |
+----+---------+
|  1 |     100 |
|  2 |     200 |
+----+---------+
2 rows in set (0.00 sec)

mysql>
```

(In this screenshot, the transaction made some updates, but then rolled back, and the updates were undone.)

Transaction Properties (ACID)

Atomicity

- A transaction is treated as a single, indivisible, logical unit of work. All operations in a transaction must be completed; if not, then the transaction is aborted

Consistency

- Constraints that hold before a transaction must also hold after it. A transaction must not violate any integrity constraints or rules that govern the database.
- (multiple users accessing the same data see the same value)

Isolation

- Multiple transactions can occur concurrently without leading to the inconsistency of the database state. The intermediate states of transactions are invisible to other transactions until they are committed. The data required by an executing transaction cannot be accessed by any other transaction until the first transaction finishes. (This ensures Consistency)

Durability

- When a transaction is complete, the changes made to the database are permanent, even if the system fails. The updates are permanent and are stored in non-volatile memory, such as a hard disk or solid-state drive.

Exercise 1

A designer brands retailer has the following tables in their database: Product, ProductStockLevel, Action. Note there are other tables which are not relevant to this question. The Action table keeps track of all sales and returns.

Product

ProdID	Brand	Description	Retail Price	PurchasePrice
G43546	Gucci	Leather mid-heel pump	1080.00	450.00

ProductStockLevel

ProdID	QtyInStock
G43546	12

Action

ActionID	ActionDateTime	ActionType	ProdID	ProdQty	ProdCost
1008	21/01/2021 10:24	Purchase	G43546	2	2160.00
1026	23/01/2021 13:28	Return	G43546	-1	1080.00

What is the problem with the following transaction?

```
START TRANSACTION;
INSERT INTO Product VALUES (G43547, "Gucci", "BAMBOO 1947 MINI TOP HANDLE BAG", 5840, 4300);
INSERT INTO ProductStockLevel VALUES (G43547, 0);
UPDATE Product SET RetailPrice = RetailPrice * 0.95;
COMMIT;
```

Exercise 2

A designer brands retailer has the following tables in their database: Product, ProductStockLevel, Action. Note there are other tables which are not relevant to this question. The Action table keeps track of all sales and returns.

Product

ProdID	Brand	Description	Retail Price	PurchasePrice
G43546	Gucci	Leather mid-heel pump	1080.00	450.00

ProductStockLevel

ProdID	QtyInStock
G43546	12

Action

ActionID	ActionDateTime	ActionType	ProdID	ProdQty	ProdCost
1008	21/01/2021 10:24	Purchase	G43546	2	2160.00
1026	23/01/2021 13:28	Return	G43546	-1	1080.00

Create an SQL transaction that records a sale of the product G43546 with the quantity being 2.

Exercise 2 - solution

Product

ProdID	Brand	Description	Retail Price	PurchasePrice
G43546	Gucci	Leather mid-heel pump	1080.00	450.00

ProductStockLevel

ProdID	QtyInStock
G43546	12

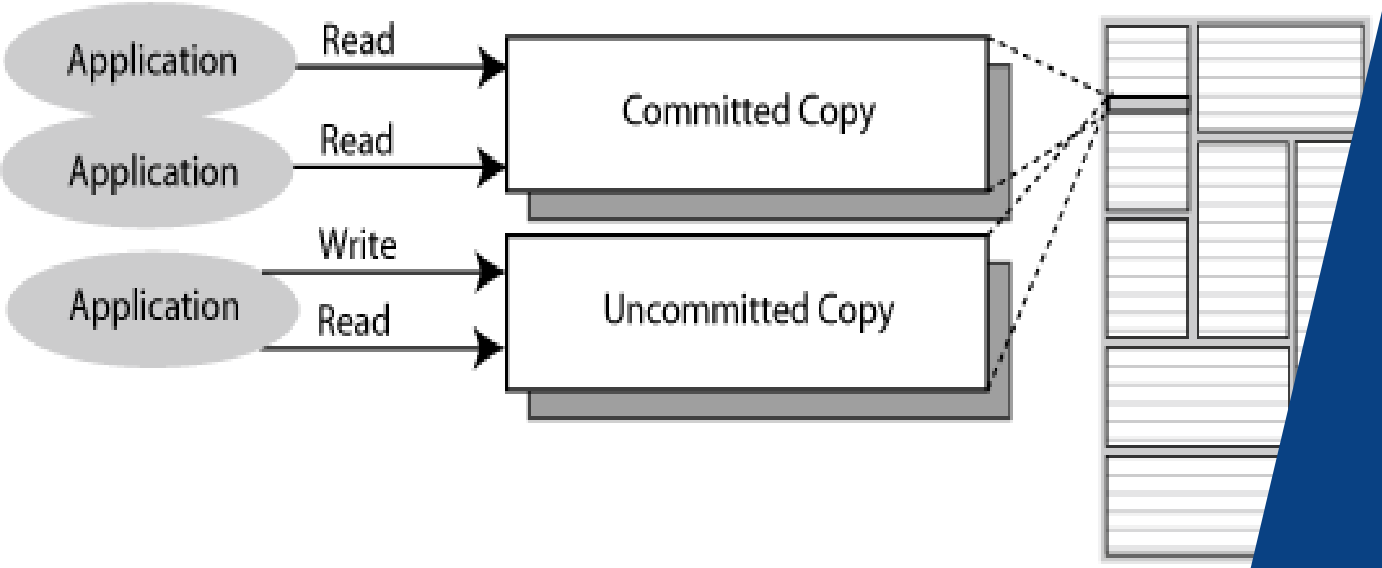
Action

ActionID	ActionDateTime	ActionType	ProdID	ProdQty	ProdCost
1008	21/01/2021 10:24	Purchase	G43546	2	2160.00
1026	23/01/2021 13:28	Return	G43546	-1	1080.00

Create an SQL transaction that records a sale of the product G43546 with the quantity being 2.

We need 2 updates: a new purchase activity in the Action table and stock update in ProductStockLevel

```
START TRANSACTION;
SET @qty = 2;
SET @price = (SELECT RetailPrice FROM Product WHERE ProdID='G43546');
SET @amount = @price*@qty;
INSERT INTO Action VALUES (DEFAULT, '2024-04-15', "Purchase", "G43546", @qty, @amount);
UPDATE ProductStockLevel SET QtyInStock = QtyInStock - @qty WHERE ProdID='G43546';
COMMIT;
```



Concurrency

What happens if there is more than one user?

Problem 2: Concurrent access

What happens if we have multiple users accessing the database at the same time…

Concurrent execution of DML against a shared database

Note that the sharing of data among multiple users is where much of the benefit of databases derives – users communicate and collaborate via shared data

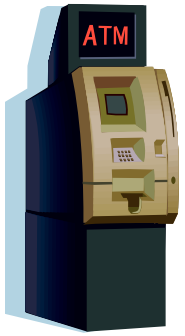
But what could go wrong?

- lost updates
- uncommitted data
- inconsistent retrievals

The Lost Update problem

Two concurrent transactions are updating the same piece of data, and one of the updates is lost because it is overwritten by the other transaction.

Alice



Read account balance (balance = $1000)

Withdraw $100 (balance = $900)

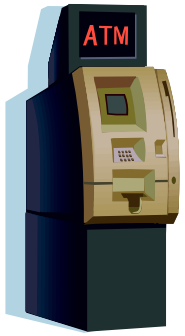
Write balance balance = $900

Time

t1a *t2a* *t3a*

t1b *t2b* *t3b*

Bob



Read account balance (balance = $1000)

Withdraw $800 (balance = $200)

Write balance balance = $200

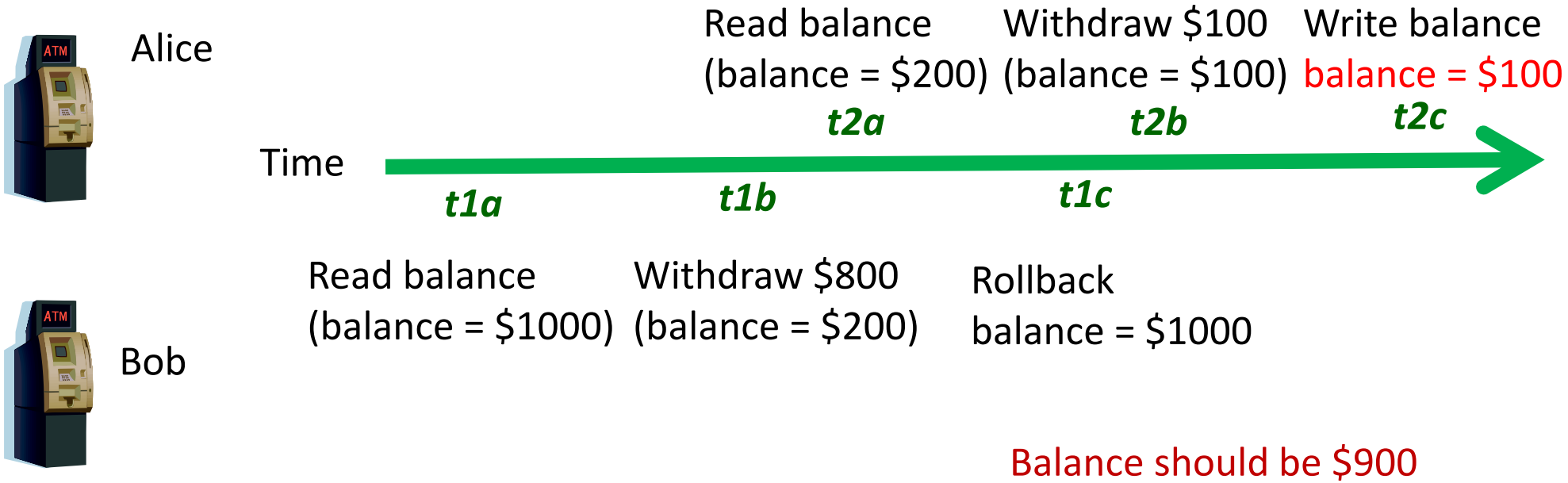
Example: Alice and Bob accessing the same account affecting its balance.

Balance should be $100

The Uncommitted Data problem

Uncommitted data occurs when two transactions execute concurrently,
and the first is rolled back after the second has already accessed the uncommitted data.

(Isolation property is violated)



The Inconsistent Retrieval problem

Occurs when one transaction calculates some aggregate functions over a set of data, while other transactions are updating the data

- Some data may be read *after* they are changed and some *before* they are changed, yielding inconsistent results

Alice	Bob
SELECT SUM(Salary) FROM Employee;	UPDATE Employee SET Salary = Salary * 1.01 WHERE EmpID = 33;
	UPDATE Employee SET Salary = Salary * 1.01 WHERE EmpID = 44;
(finishes calculating sum)	COMMIT;

Example: Inconsistent Retrieval

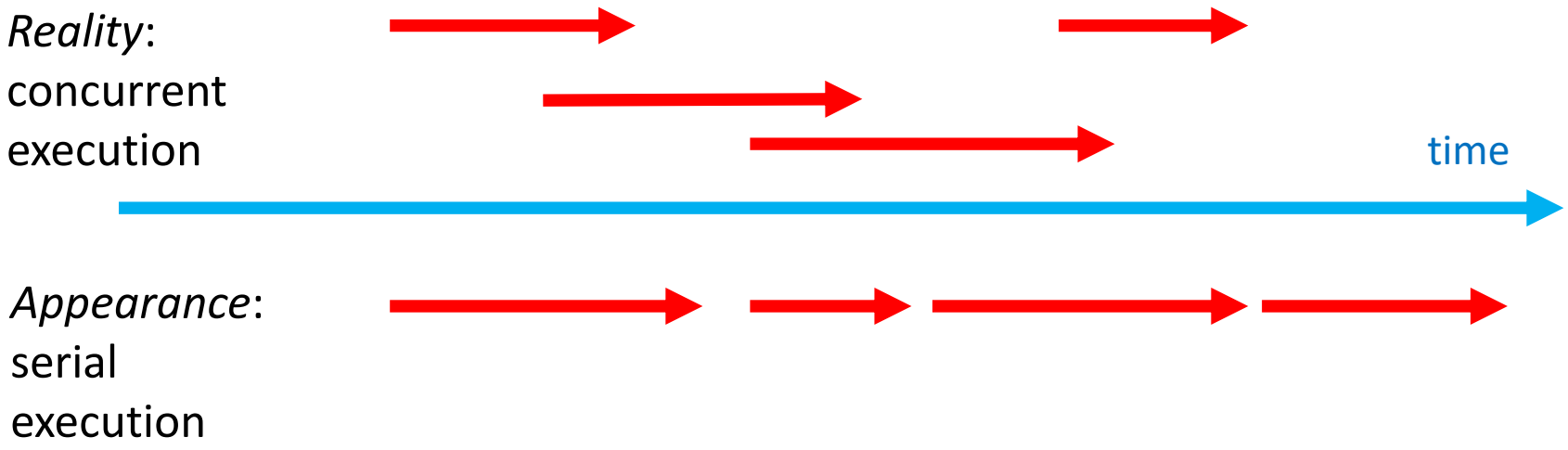
Time	Trans-action	Action	Value	T1 SUM	Comment
1	T1	Read Salary for EmpID 11	10,000	10,000	
2	T1	Read Salary for EmpID 22	20,000	30,000	
3	T2	Read Salary for EmpID 33	30,000		
4	T2	Salary = Salary * 1.01			
5	T2	Write Salary for EmpID 33	30,300		
6	T1	Read Salary for EmpID 33	30,300	60,300	*after* update
7	T1	Read Salary for EmpID 44	40,000	100,300	*before* update
8	T2	Read Salary for EmpID 44	40,000		
9	T2	Salary = Salary * 1.01			
10	T2	Write Salary for EmpID 44	40,400		
11	T2	COMMIT			
12	T1	Read Salary for EmpID 55	50,000	150,300	
13	T1	Read Salary for EmpID 66	60,000	210,300	

we want either
before $210,000 or
after $210,700

Serializability

Transactions ideally are “serializable”

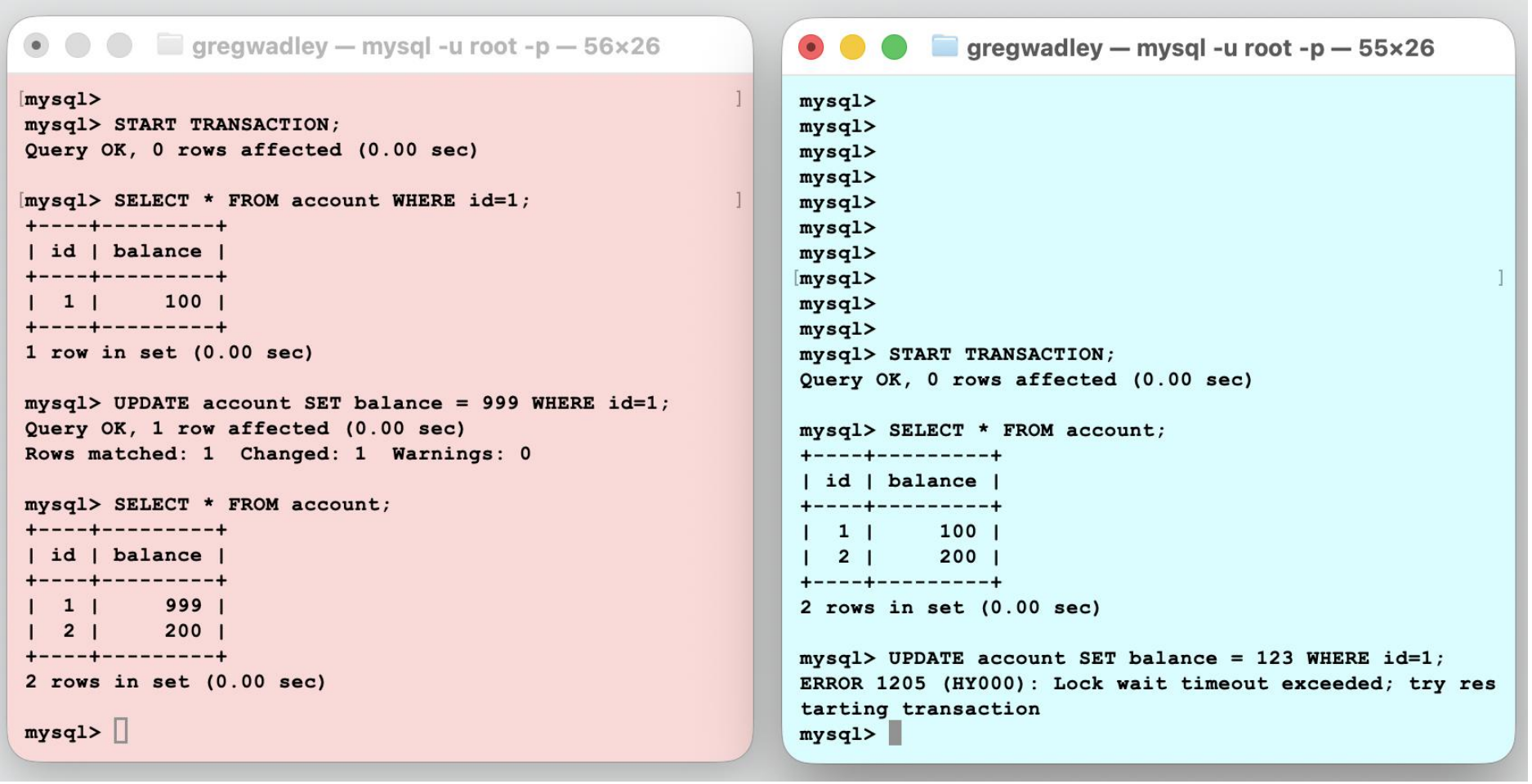
- Multiple, concurrent transactions *appear as if* they were executed one after another
- Ensures that the concurrent execution of several transactions yields consistent results



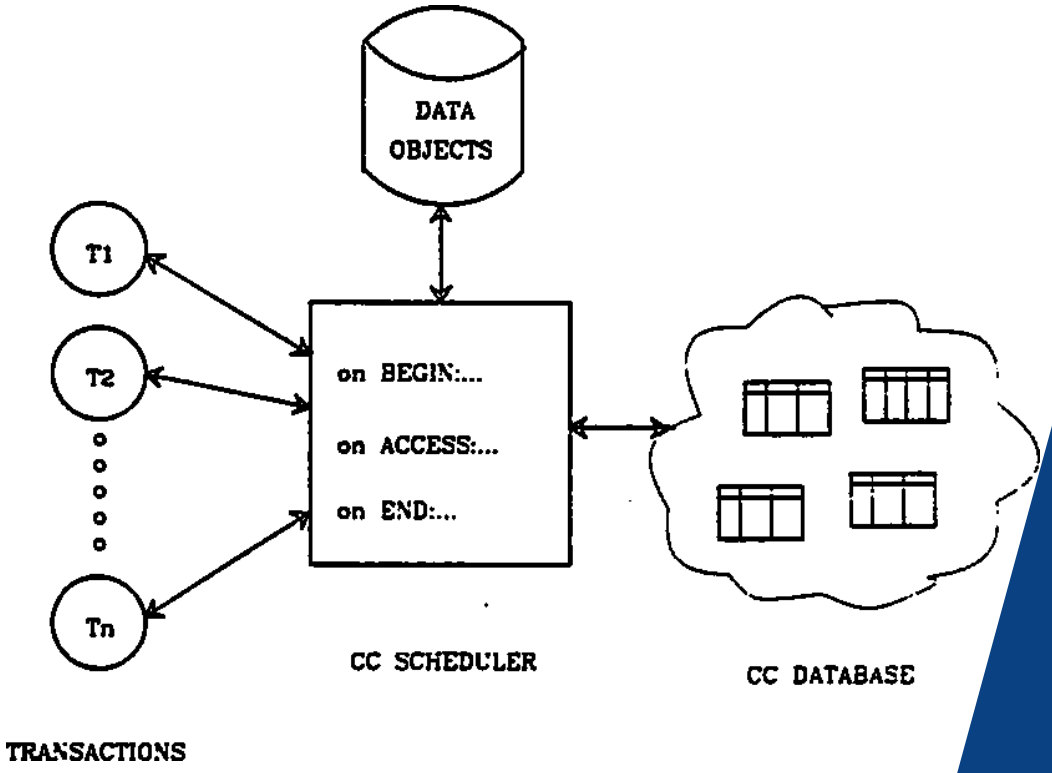
Scheduler – special DBMS process that schedules in which order operations within concurrent transactions execute, ensuring serializability and isolation.

True serial execution (i.e. no concurrency) is very expensive!

Demo: Transaction as solution to concurrency



(In this screenshot, red locked a row and blocked blue. Eventually blue timed out.)



Concurrency Control Methods

Locking
Timestamps
Optimistic Method

Concurrency control methods

To achieve efficient execution of transactions, the DBMS creates a schedule of read and write operations for concurrent transactions.
Interleaves the execution of operations, based on concurrency control algorithms such as locking and time stamping.

Several methods of concurrency control

- *Locking* is the main method used
- Time Stamping
- Optimistic Methods

Concurrency Control with Locking

Lock

- Guarantees exclusive use of a data item to a current transaction
  - T1 acquires a lock prior to data access; the lock is released when the transaction is complete
  - T2 does not have access to data item currently being used by T1
  - T2 has to wait until T1 releases the lock
- Required to prevent another transaction from reading inconsistent data

Lock manager

- Responsible for assigning and policing the locks used by the transactions

Question: at what granularity should we apply locks?

Lock Granularity: options

Database-level lock

- Entire database is locked
- Good for batch processing but unsuitable for multi-user DBMSs
- T1 and T2 can not access the same database concurrently even if they use different tables
- (SQLite, Access)

Table-level lock

- Entire table is locked - as above but not quite as bad
- Good when updating most of the rows in a table
- T1 and T2 can access the same database concurrently as long as they use different tables
- Can cause bottlenecks, even if transactions want to access different parts of the table and would not interfere with each other
- Not suitable for highly multi-user DBMSs

Lock Granularity: options

Page-level lock

- An entire disk page is locked (a table can span several pages and each page can contain several rows of one or more tables)

Row-level lock

- Allows concurrent transactions to access different rows of the same table, even if the rows are located in the same page
- Improves data availability but with high overhead (each row has a lock that must be read and written to)
- Currently the most popular approach (MySQL, Oracle)
- Higher overhead than Page level lock
- Automatically escalated to page level if several rows in the same page need to be locked.

Field-level lock

- Allows concurrent transactions to access the same row, as long as they access different attributes within that row
- A user may see some columns but not others (e.g. in online banking transactions are displayed but running balance not available)
- Most flexible lock but requires an extremely high level of overhead
- Not commonly used

Types of Locks

Binary Locks

- has only two states: locked (1) or unlocked (0)
  – Every transaction will apply a lock and then terminate a lock (release the locked data)
- eliminates “Lost Update” problem
  – the lock is not released until the statement is completed
- considered too restrictive to yield optimal concurrency, as it locks even for two READs (when no update is being done)

The alternative is to allow both *Shared* and *Exclusive* locks

- often called Read and Write locks

https://medium.com/inspiredbrilliance/what-are-database-locks-1aff9117c290#:~:text=Row%2Dlevel%20locks%20are%20the,to%20free%20up%20some%20memory

Shared and Exclusive Locks

Exclusive lock

- Access is reserved for the transaction that locked the object
- Must be used when transaction intends to WRITE
- Granted if and only if no other locks are held on the data item
- In MySQL: “SELECT … FOR UPDATE” or “LOCK TABLES … WRITE”;

Shared lock (also called Read lock)

- Other transactions are also granted Read access
- Issued when a transaction wants to READ data, and no Exclusive lock is held on that data item
  - multiple transactions can each have a shared lock on the same data item if they are all just reading it
- In MySQL: “SELECT … LOCK FOR SHARE” or “LOCK TABLES … READ;

https://www.geeksforgeeks.org/lock-based-concurrency-control-protocol-in-dbms/

Basic Lock Compatibility Matrix

	S	X
S	✔	✗
X	✗	✗

S – shared
X - exclusive

If the transaction T1 is holding a shared lock in data item A, then the control manager can grant the shared lock to transaction T2 as compatibility is TRUE, but it cannot grant the exclusive lock.

In simple words, if transaction T1 is reading a data item A, then same data item A can be read by another transaction T2 but cannot be written by another transaction.

Similarly, if an exclusive lock (i.e. lock for read and write operations) is held on the data item in some transaction then no other transaction can acquire a lock, neither Shared nor Exclusive lock.

MySQL: FOR UPDATE versus FOR SHARE

- FOR SHARE does not prevent another transaction from reading the same row that was locked. FOR UPDATE prevents other locking reads of the same row.

Source: https://www.geeksforgeeks.org/difference-between-shared-lock-and-exclusive-lock/

Deadlock

Condition that occurs when two transactions wait for each other to unlock data

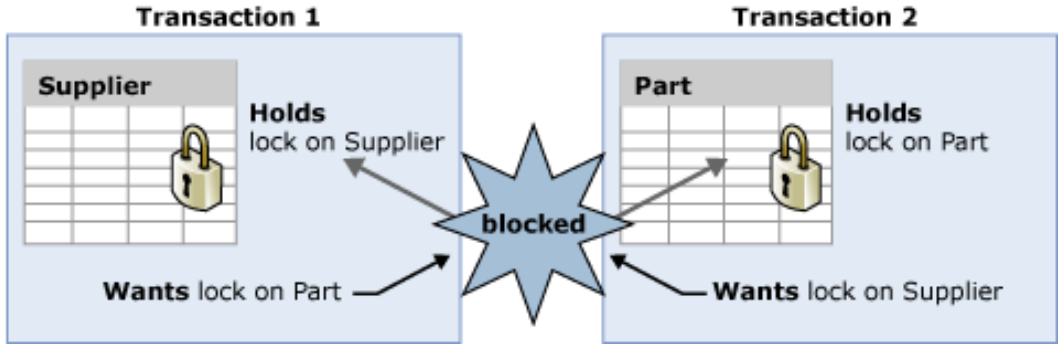
- T1 locks data item X, then wants Y
- T2 locks data item Y, then wants X
- Each waits to get a data item which the other transaction is already holding
- Could wait forever if not dealt with

Can happen with several transactions being in a circular lock

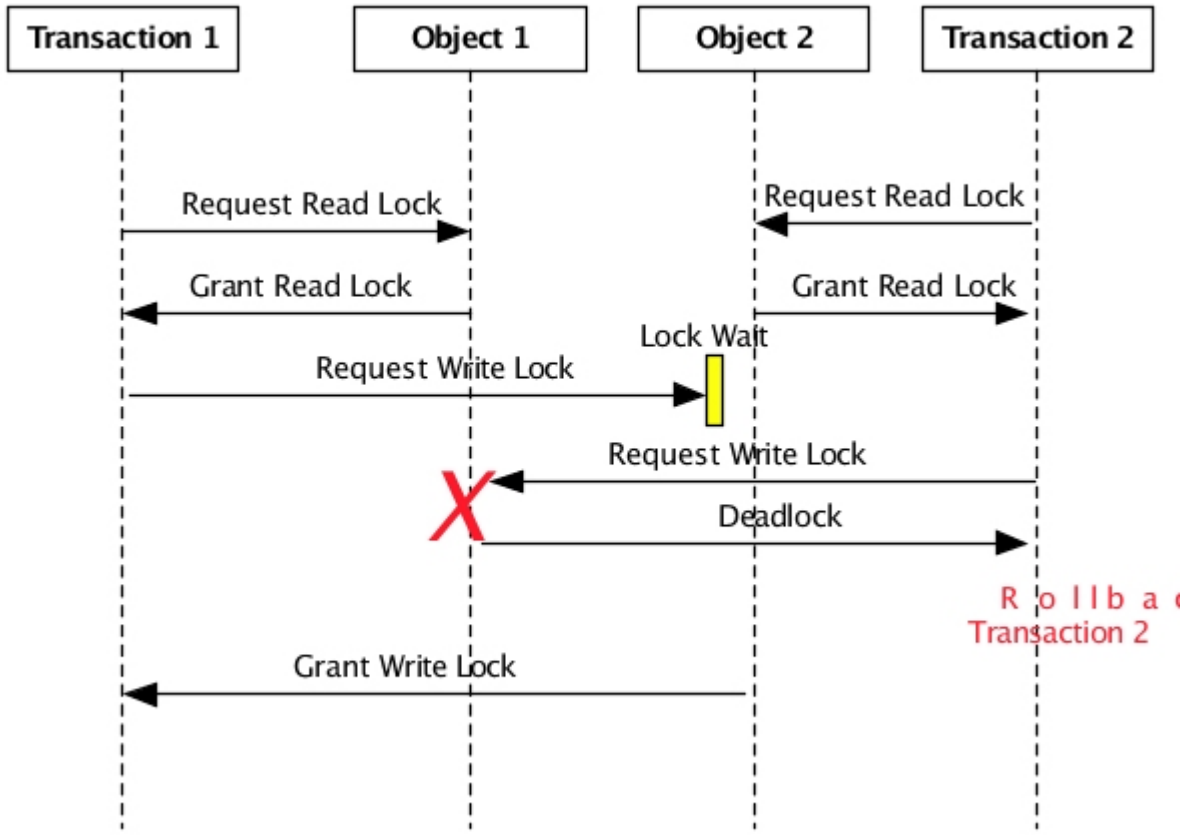
- T1 waits for T2, T2 waits for T3, T3 waits for T1

Deadlock can only happen with exclusive locks

Deadlocks cannot happen with shared locks



Deadlock example



1.Transaction 1 requests, and is granted, a read lock on Object 1.

2.Transaction 2 requests, and is granted, a read lock on Object 2.

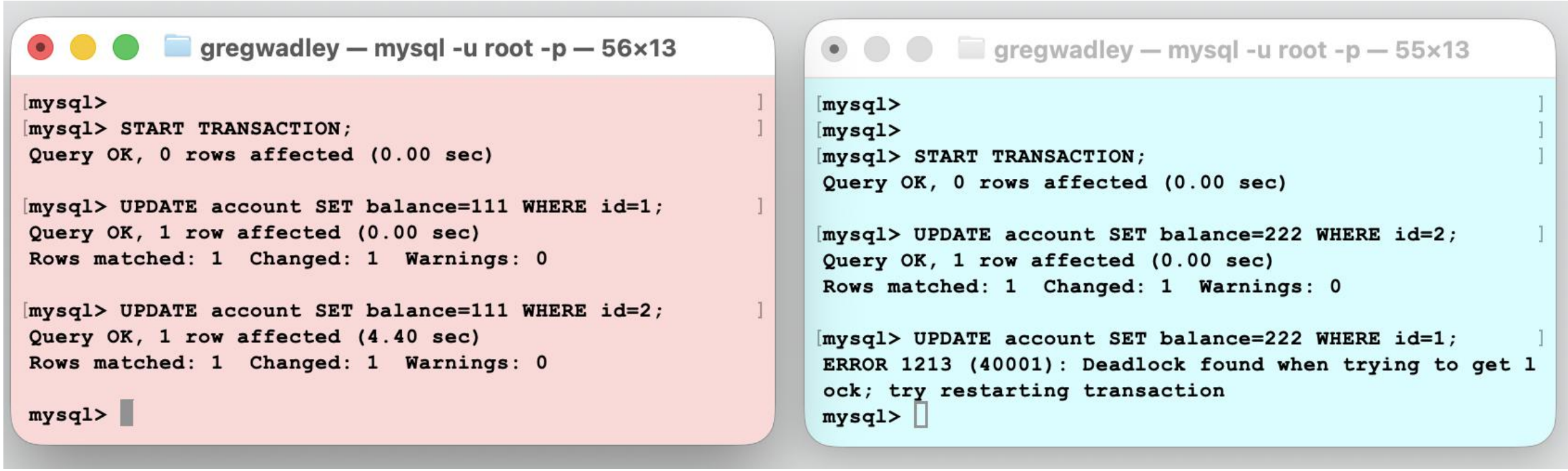
3.Transaction 1 requests, but is not granted, a write lock on Object 2. The write lock is not granted because of the read lock held on Object 2 by Transaction 2. Objects cannot be modified while other transactions are reading the object.

4.Transaction 2 requests, but is not granted, a write lock on Object 1. This is a deadlock because both transactions would block indefinitely waiting for the other to complete. Transaction 2 is chosen as the *victim* and rolled back.

5.Transaction 1 is granted the requested write lock on Object 2 because Transaction 2's read lock on Object 2 was released when Transaction 2 was rolled back.

Source: https://docs.tibco.com/pub/ast/2.5.11/doc/html/architectsguide/ch05s04.html

Demo: Transaction leading to deadlock



(In this screenshot, red locked a row and blue locked a row. Then, red and blue each tried to access the rows that the other had locked. MySQL detected a deadlock.)

Dealing with Deadlocks

Techniques to avoid, detect and resolve deadlocks:

- Timeout mechanisms
- Prevention
  - preventing deadlocks by careful design of transactions, such as always acquiring locks in the same order or releasing locks as soon as possible.
- Detection
  - detection algorithms, which periodically scan the transaction log for deadlock cycles and then choose a transaction to abort to resolve the deadlock.
- Avoidance
  - Avoiding circular wait conditions
  - Using an algorithm to identify whether running some transactions could result in a deadlock

Strict Two-Phase Locking (Strict 2PL)

Two Phases

1. A transaction T must *obtain all locks* before it can start reading or writing data (growing phase). No unlocking can happen if another lock is required.
2. All locks held by transaction T are released when the transaction is completed (shrinking phase).

If two transactions are accessing completely different parts of the database, they concurrently obtain the locks and proceed.

If two transactions T_1, T_2 are accessing the same object and one wants to modify it, their actions are ordered serially – all actions of one transaction T_1 must complete before the transaction T_2 can proceed.

https://www.tutorialspoint.com/dbms/dbms_concurrency_control.htm

Exercise

Product

ProdID	Brand	Description	Retail Price	PurchasePrice
G43546	Gucci	Leather mid-heel pump	1080.00	450.00

ProductStockLevel

ProdID	QtyInStock
G43546	12

Action

ActionID	ActionDateTime	ActionType	ProdID	ProdQty	ProdCost
1008	21/01/2021 10:24	Purchase	G43546	2	2160.00
1026	23/01/2021 13:28	Return	G43546	-1	1080.00

The SQL transaction below records a sale of the product G43546 with the quantity being 2.

Assuming that pessimistic locking with the two-phase locking protocol is being used with row-level lock granularity, create a chronological list of the locking, unlocking, and data manipulation activities that would occur during the complete processing of the transaction.

```
START TRANSACTION;
SET @qty = 2;
SET @price = (SELECT RetailPrice FROM Product WHERE ProdID='G43546');
SET @amount = @price*@qty;
INSERT INTO Action VALUES (DEFAULT, '2024-04-15', "Purchase", "G43546", @qty, @amount);
UPDATE ProductStockLevel SET QtyInStock = QtyInStock - @qty WHERE ProdID='G43546';
COMMIT;
```

Exercise - solution

```
START TRANSACTION;
SET @qty = 2;
SET @price = (SELECT RetailPrice FROM Product WHERE ProdID='G43546');
SET @amount = @price*@qty;
INSERT INTO Action VALUES (1028, '2024-04-15', "Purchase", "G43546", @qty, @amount);
UPDATE ProductStockLevel SET QtyInStock = QtyInStock - @qty WHERE ProdID='G43546';
COMMIT;
```

Time	Action
1	Lock Product row where ProdID = G43546
2	Lock Action
3	Lock ProductInStock row where ProdID = G43546
4	@qty = 2
5	@price = get RetailPrice from Product for ProdID = G43546
6	Insert row <DEFAULT> INTO Action
7	Update ProductInStock QtyInStock decreased by @qty for ProdID = G43546
8	Unlock Product row where ProdID = G43546
9	Unlock Action
10	Unlock ProductInStock row where ProdID = G43546

Alternatives to locking: multi-version control

With simpler locking mechanisms, as depicted in the previous slide, writers block readers and other writers and readers block writers.

With more advanced multi-version concurrency control (MySQL is a DBMS that implements this), writers need not completely block readers and vice versa.

When a transaction updates a value, the DBMS creates a new version of the value.

Readers can see ‘old’ version of record while writer transaction is under way.

More details (optional)

https://medium.com/@ajones1_999/understanding-mysql-multiversion-concurrency-control-6b52f1bd5b7e

Alternatives to locking: timestamp, optimistic

Timestamp

- Assigns a global unique timestamp to each transaction (how ‘old’ or ‘new’ the transaction is)
- DBMS tracks which transaction last read, and which transaction last wrote, the data
- When a transaction wants to read or write, the DBMS compares its timestamp with the timestamps already attached to the item and decides whether to allow access
- Old T can’t read data changed by a new T. Old T can’t write data read/changed by a new T.

Optimistic

- Based on the assumption that the majority of database operations *do not* conflict
- Transaction is executed without restrictions or checking
- Then when it is ready to commit, the DBMS checks whether any of the data it read has been altered – if so, rollback

Logging transactions

If transaction cannot be completed, it must be aborted and any changes rolled back .

To enable this, DBMS tracks all updates to data

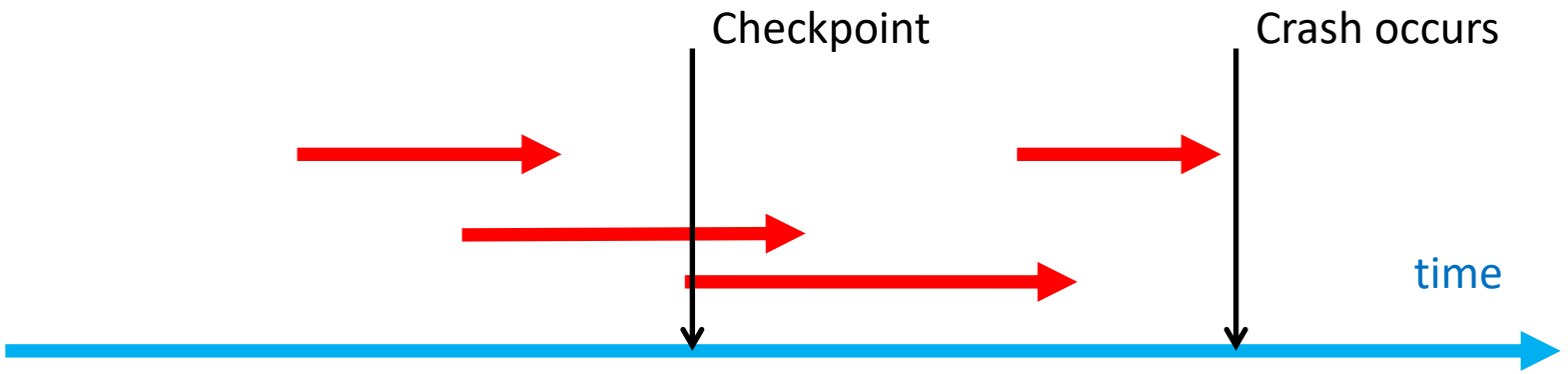
This *transaction log* contains:

- a record for the beginning of the transaction
- for each SQL statement
  - operation being performed (update, delete, insert)
  - objects affected by the transaction
  - “before” and “after” values for updated fields
  - pointers to previous and next transaction log entries
- the ending (COMMIT) of the transaction

Transaction log

Also provides the ability to restore a corrupted database

If a system failure occurs, the DBMS will examine the log for all uncommitted or incomplete transactions and it will restore the database to its previous state



Example transaction log

TRL ID	TRX NUM	PREV PTR	NEXT PTR	OPERATION	TABLE	ROW ID	ATTRIBUTE	BEFORE VALUE	AFTER VALUE
341	101	Null	352	START	****Start Transaction				
352	101	341	363	UPDATE	PRODUCT	54778-2T	PROD_QOH	45	43
363	101	352	365	UPDATE	CUSTOMER	10011	CUST_BALANCE	615.73	675.62
365	101	363	Null	COMMIT	**** End of Transaction				
397	106	Null	405	START	****Start Transaction				
405	106	397	415	INSERT	INVOICE	1009			1009,10016, ...
415	106	405	419	INSERT	LINE	1009,1			1009,1, 89-WRE-Q,1, ...
419	106	415	427	UPDATE	PRODUCT	89-WRE-Q	PROD_QOH	12	11
423				CHECKPOINT					
427	106	419	431	UPDATE	CUSTOMER	10016	CUST_BALANCE	0.00	277.55
431	106	427	457	INSERT	ACCT_TRANSACTION	10007			1007,18-JAN-2004, ...
457	106	431	Null	COMMIT	**** End of Transaction				
521	155	Null	525	START	****Start Transaction				
525	155	521	528	UPDATE	PRODUCT	2232/QWE	PROD_QOH	6	26
528	155	525	Null	COMMIT	**** End of Transaction				
* * * * * C *R*A* S* H * * * *									

What's examinable

Why we need user-defined transactions

Properties of transactions

- ACID

How to use transactions

- BEGIN; START TRANSACTION; COMMIT; ROLLBACK;

Concurrent access to data

- Concurrent access strategies

Locking and deadlocking

- Types of Locks (Binary; Shared)

Database recovery

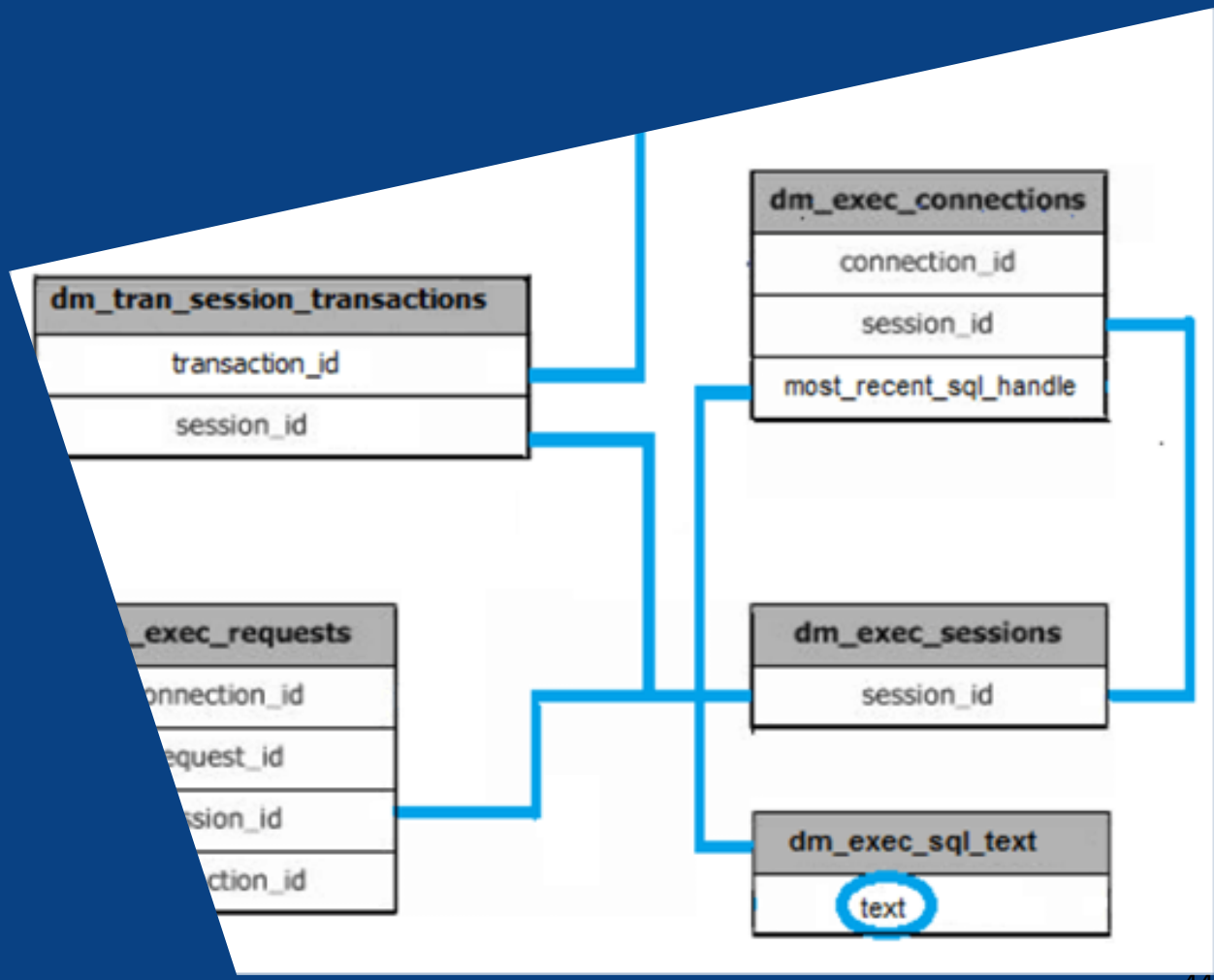
- Fundamentals of transaction recovery

*** All material is examinable – these are the suggested key skills you would need to demonstrate in an exam scenario**

THE UNIVERSITY OF MELBOURNE

Transactions and Concurrency

Database Systems & Information Modelling
INFO90002



Week 7 – Database transactions
Presented by Greg Wadley
Content by: Dr Tanya Linden, Dr Renata Borovica-Gajic, David Eccles, Dr Simon D'Alfonso, Dr Greg Wadley